

Java Backend Architecture

Interview Series

Chapter 11.7

Application-Level Conflict Resolution

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

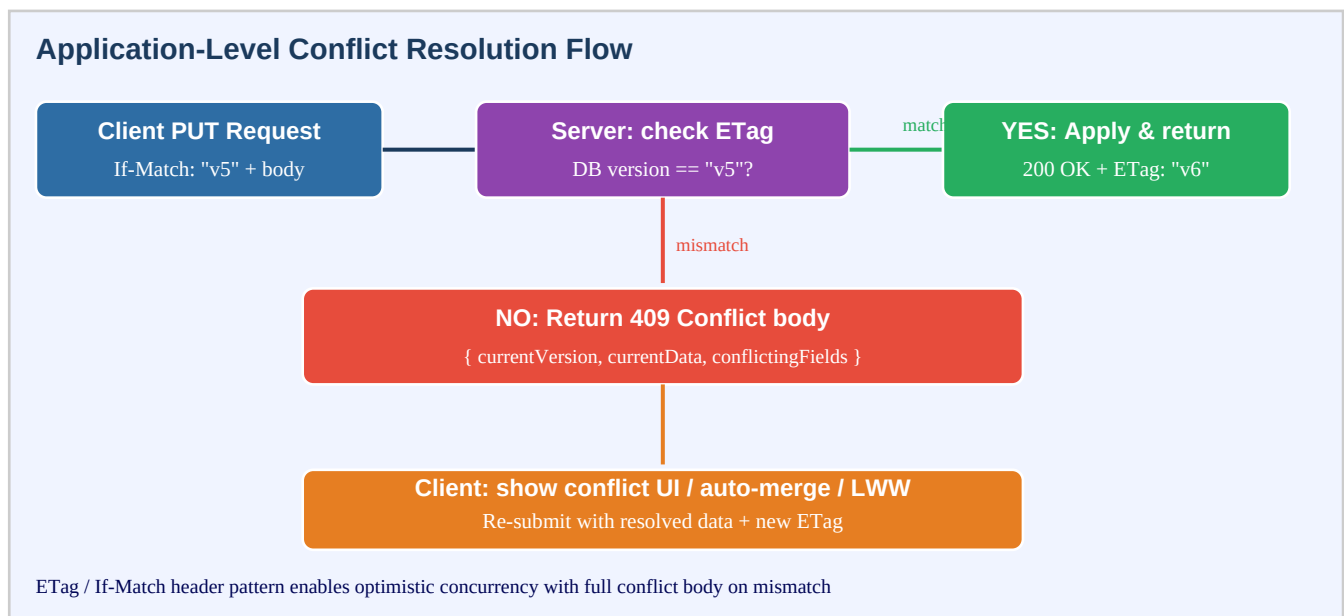
Chapter 11.7 – Application-Level Conflict Resolution

Introduction

Application-level conflict resolution is the discipline of designing APIs and backend logic that gracefully detect, surface, and resolve situations where two or more concurrent requests attempt to modify the same resource in incompatible ways. Unlike database-level pessimistic locking, application-level approaches use optimistic concurrency — clients carry a version identifier (ETag, version number, or timestamp) and the server rejects writes that arrive with stale versions by returning HTTP 409 Conflict with a rich response body that includes the current server state, the conflicting fields, and guidance for resolution. This enables responsive UIs while maintaining data integrity.

The choice of merge strategy — last-write-wins, 3-way merge, or field-level merge — significantly impacts user experience and data integrity. Last-write-wins is simple but silently discards concurrent edits, making it unsuitable for collaborative data. Three-way merge (like git) uses a common ancestor to combine divergent changes safely. Field-level merge resolves each attribute independently, allowing non-overlapping edits from concurrent users to coexist without conflict. The booking hold pattern extends these concepts to reservation scenarios, using a HELD intermediate state with a TTL timer to prevent double-booking without long-held pessimistic locks.

Conflict Resolution Flow



Merge Strategy Comparison

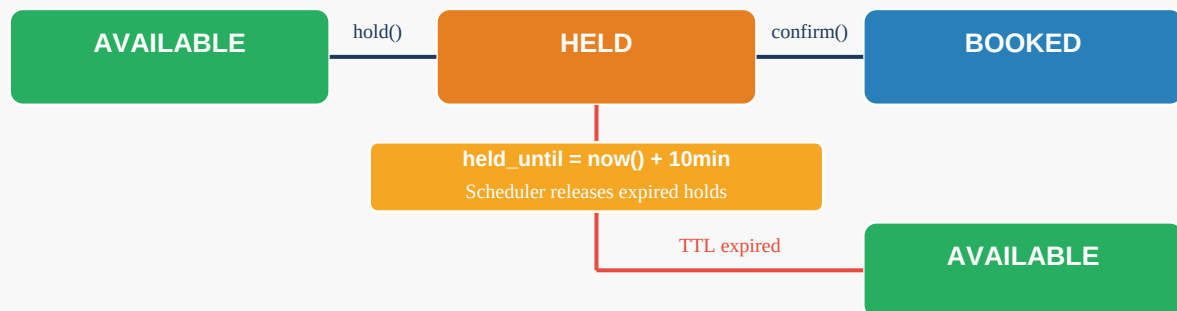
Merge Strategy Comparison

	Last-Write-Wins	3-Way Merge	Field-Level Merge
	Highest timestamp / version Loses concurrent edits Simple but lossy.	base + client-change + serve Like git merge Complex but safe.	Each field resolved independ Client.name + Server.price =
Simplicity:	★★★	★★	★★★
Data Safety:	★	★★★	★★★
Conflict UX:	None	Diff UI	Per-field UI

Field-level merge preferred for collaborative editing; LWW acceptable for append-only or low-conflict scenarios

Booking Hold Pattern – HELD Status with TTL

Booking Hold Pattern – HELD Status with TTL Timer



hold() uses SELECT FOR UPDATE to prevent double-hold; TTL scheduler uses UPDATE WHERE status=HELD AND held_until < now()
confirm() validates hold still belongs to same user session before transitioning to BOOKED

Key Definitions

HTTP 409 Conflict	Status code returned when a request conflicts with the current resource state; body must include current version and conflicting fields
ETag	Entity Tag: an opaque version string (hash or version number) sent with responses; clients echo it in If-Match for conditional writes
If-Match header	Request header carrying the client's known ETag; server rejects (412 or 409) if DB version differs
Optimistic Concurrency	Assume conflicts are rare; allow concurrent reads; detect and reject conflicting writes at commit time using version check
Pessimistic Locking	Lock resources before reading; prevents conflicts but reduces throughput and risks deadlocks
Last-Write-Wins (LWW)	Conflict resolved by keeping the change with the highest timestamp or version; concurrent edits from the loser are silently discarded

3-Way Merge	Merge algorithm using base version + client changes + server changes; non-overlapping changes merged automatically; overlapping flagged
Field-Level Merge	Each field resolved independently: fields changed only by client keep client value; fields changed only by server keep server value; both-changed = conflict
Booking Hold Pattern	Reserve a resource by transitioning it to HELD status with a TTL; prevents double-booking without long locks; scheduler releases expired holds
TTL Timer	Time-to-live on a hold; held_until timestamp in DB; background job releases holds where held_until < now()
Idempotency Key	Client-generated UUID in X-Idempotency-Key header; server stores processed results; duplicate requests return cached response without re-processing
Conflict Body Schema	Structured JSON in 409 body: { error, currentVersion, currentData, conflictingFields[], yourData, resolution_hint }
OptimisticLockException	JPA/Hibernate exception thrown when @Version field mismatch detected at flush time; mapped to HTTP 409 via @ExceptionHandler
@Version	JPA annotation on a Long/Integer field; Hibernate auto-increments and checks version on every UPDATE; throws OptimisticLockException on mismatch
Merge UI Pattern	Frontend pattern: show diff view with current server value vs client value; let user pick or edit the final resolution; re-submit with resolved data

Merge Strategy Comparison

Strategy	Conflict Detection	Data Loss Risk	Complexity	Best Use Case
Last-Write-Wins	Timestamp/version	High (concurrent edits lost)	Low	Append-only, low-conflict data
First-Write-Wins	Version check → reject later winners	High (rejects losers)	Low	Inventory reservations, bookings
3-Way Merge	Common ancestor diff	Low (only true conflicts)	High	Document/config editing (git-style)
Field-Level Merge	Per-field version tracking	Very Low	Medium	CRM records, shared forms
Booking Hold	HELD status + TTL	None	Medium	Ticket/seat/room reservations
CRDT-based	Mathematical merge (commutative)	None	Very High	Real-time collaborative editing

Code Examples

1. HTTP 409 Conflict Response Body Schema

```
// 409 response body - include everything the client needs to resolve the conflict { "error":
"CONFLICT", "message": "Resource was modified by another user. Please review and resubmit.",
"currentVersion": "v7", "currentData": { "id": "booking-123", "status": "CONFIRMED", "price":
299.99, "updatedAt": "2025-05-30T14:22:00Z", "updatedBy": "user-456" }, "yourVersion": "v5",
"yourData": { "price": 259.99, "status": "CONFIRMED" }, "conflictingFields": ["price"],
"nonConflictingFields": [], "resolutionHint": "FIELD_LEVEL_MERGE_POSSIBLE" } //
conflictingFields = fields where both you and server changed the value // nonConflictingFields
= fields only you changed (safe to auto-apply) // resolutionHint: FIELD_LEVEL_MERGE_POSSIBLE |
MANUAL_REVIEW_REQUIRED | AUTO_MERGED
```

2. @ExceptionHandler for OptimisticLockException → 409

```
@RestControllerAdvice public class ConflictExceptionHandler {
    @ExceptionHandler(OptimisticLockException.class) @ResponseStatus(HttpStatus.CONFLICT) public
    ConflictResponse handleOptimisticLock( OptimisticLockException ex, HttpServletRequest request)
    { // Extract entity from exception to get current DB state Object entity = ex.getEntity(); //
    may be null depending on JPA provider return ConflictResponse.builder() .error("CONFLICT")
    .message("Resource modified concurrently. Review current state and resubmit.")
    .resolutionHint("Fetch current resource with GET, merge changes, resubmit with current ETag")
    .build(); } // Also handle Spring's wrapper:
    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
    @ResponseStatus(HttpStatus.CONFLICT) public ConflictResponse handleSpringOptimisticLock(
    ObjectOptimisticLockingFailureException ex) { String entityType = ex.getPersistentClassName();
    Object entityId = ex.getIdentifier(); // Reload current state for richer 409 body: // Object
    current = repository.findById(entityId).orElse(null); return ConflictResponse.builder()
    .error("CONFLICT") .currentEntityType(entityType) .currentEntityId(String.valueOf(entityId))
    .message("Concurrent modification detected on " + entityType + " id=" + entityId) .build(); } }
    // JPA entity with @Version: @Entity public class Booking { @Id private String id; @Version
    private Long version; // auto-incremented by Hibernate on each UPDATE private String status;
    private BigDecimal price; }
```

3. REST API with ETag / If-Match Optimistic Concurrency

```
@RestController @RequestMapping("/bookings") public class BookingController {
    @GetMapping("/{id}") public ResponseEntity<BookingDto> get(@PathVariable String id) { Booking
    b = bookingService.findById(id); String etag = "v" + b.getVersion() + ""; return
    ResponseEntity.ok() .eTag(etag) // ETag: "v5" .body(toDto(b)); } @PutMapping("/{id}") public
    ResponseEntity<?> update( @PathVariable String id, @RequestHeader(value = "If-Match", required
    = false) String ifMatch, @RequestBody BookingUpdateRequest req) { if (ifMatch == null) { return
    ResponseEntity.status(428).body("Precondition Required: send If-Match header"); } // Strip
    quotes: "v5" -> v5 -> 5 long clientVersion = Long.parseLong(ifMatch.replaceAll("[\"v]", ""));
    try { Booking updated = bookingService.update(id, req, clientVersion); String newEtag = "v" +
    updated.getVersion() + ""; return ResponseEntity.ok().eTag(newEtag).body(toDto(updated)); }
    catch (ObjectOptimisticLockingFailureException ex) { Booking current =
    bookingService.findById(id); List<String> conflicting = detectConflictingFields(req, current);
    return ResponseEntity.status(409).body(ConflictResponse.of(current, req, conflicting)); } }
```

4. Booking Hold Pattern Implementation

```
@Service @Transactional public class BookingHoldService { private static final Duration
    HOLD_TTL = Duration.ofMinutes(10); // Step 1: Acquire hold (pessimistic lock to prevent race)
    public HoldResponse holdSeat(String seatId, String userId) { // SELECT ... FOR UPDATE prevents
    concurrent holds on same seat Seat seat = seatRepo.findByIdForUpdate(seatId) .orElseThrow(() ->
    new SeatNotFoundException(seatId)); if (seat.getStatus() != SeatStatus.AVAILABLE) { throw new
    SeatNotAvailableException(seatId, seat.getStatus()); } seat.setStatus(SeatStatus.HELD);
    seat.setHeldBy(userId); seat.setHeldUntil(Instant.now().plus(HOLD_TTL)); seatRepo.save(seat);
    return new HoldResponse(seatId, seat.getHeldUntil()); } // Step 2: Confirm booking within TTL
    public Booking confirmHold(String seatId, String userId, PaymentDetails payment) { Seat seat =
    seatRepo.findByIdForUpdate(seatId).orElseThrow(); if (seat.getStatus() != SeatStatus.HELD)
    throw new HoldExpiredException(seatId); if (!userId.equals(seat.getHeldBy())) throw new
    HoldOwnershipException(seatId); if (Instant.now().isAfter(seat.getHeldUntil())) throw new
    HoldExpiredException(seatId); seat.setStatus(SeatStatus.BOOKED); seat.setHeldUntil(null);
    return bookingRepo.save(new Booking(seat, userId, payment)); } // Scheduled: release expired
    holds @Scheduled(fixedDelay = 60_000) @Transactional public void releaseExpiredHolds() { int
    released = seatRepo.releaseExpiredHolds(Instant.now()); // UPDATE seats SET status=AVAILABLE,
    held_by=null, held_until=null // WHERE status=HELD AND held_until < :now log.info("Released {}
```

```
expired holds", released); } }
```

5. Idempotency Key Deduplication

```
// Table: idempotency_keys(key VARCHAR PK, response_body TEXT, // status_code INT, created_at
// TIMESTAMP, expires_at TIMESTAMP) @Component public class IdempotencyFilter extends
OncePerRequestFilter { @Override protected void doFilterInternal(HttpServletRequest request,
HttpServletRequest response, FilterChain chain) throws ServletException, IOException { String
idempotencyKey = request.getHeader("X-Idempotency-Key"); if (idempotencyKey == null ||
!isWriteMethod(request.getMethod())) { chain.doFilter(request, response); return; } // Check
for existing result Optional<IdempotencyRecord> existing =
idempotencyRepo.findByKey(idempotencyKey); if (existing.isPresent()) { IdempotencyRecord rec =
existing.get(); // Replay cached response – do NOT re-execute business logic
response.setStatus(rec.getStatusCode()); response.setContentType("application/json");
response.getWriter().write(rec.getResponseBody()); return; } // Capture response
ContentCachingResponseWrapper cached = new ContentCachingResponseWrapper(response);
chain.doFilter(request, cached); cached.copyBodyToResponse(); // Persist result for future
duplicate requests String body = new String(cached.getContentAsByteArray(),
StandardCharsets.UTF_8); idempotencyRepo.save(IdempotencyRecord.builder() .key(idempotencyKey)
.responseBody(body) .statusCode(cached.getStatus())
.expiresAt(Instant.now().plus(Duration.ofHours(24))) .build()); } }
```

6. Field-Level Merge Logic

```
// Field-level merge: compare base (common ancestor), client, server versions public class
FieldLevelMerger<T> { public MergeResult<T> merge(T base, T client, T server) { Map<String,
Object> baseMap = toMap(base); Map<String, Object> clientMap = toMap(client); Map<String,
Object> serverMap = toMap(server); Map<String, Object> merged = new LinkedHashMap<>();
List<String> conflicts = new ArrayList<>(); for (String field : baseMap.keySet()) { Object
baseVal = baseMap.get(field); Object clientVal = clientMap.get(field); Object serverVal =
serverMap.get(field); boolean clientChanged = !Objects.equals(baseVal, clientVal); boolean
serverChanged = !Objects.equals(baseVal, serverVal); if (!clientChanged && !serverChanged) {
merged.put(field, baseVal); // No change: keep base } else if (clientChanged && !serverChanged)
{ merged.put(field, clientVal); // Only client changed: use client } else if (!clientChanged) {
merged.put(field, serverVal); // Only server changed: use server } else if
(Objects.equals(clientVal, serverVal)) { merged.put(field, clientVal); // Both changed to same
value: fine } else { conflicts.add(field); // True conflict: both changed differently
merged.put(field, serverVal); // Default to server; UI will ask user } } return new
MergeResult<>(fromMap(merged), conflicts); } } // If conflicts is empty: auto-merge succeeded,
resubmit merged object // If conflicts non-empty: return 409 with merged object and conflicting
field list // Client UI shows diff for conflicting fields only; user picks resolution
```

Interview Questions & Answers

Q1. What information should a 409 Conflict response body contain?

A well-designed 409 body must give the client everything it needs to resolve the conflict without an additional round-trip: (1) error code and human-readable message. (2) `currentVersion` — the ETag/version of the resource as it currently exists in the database. (3) `currentData` — the full current resource so the client can display it to the user. (4) `yourVersion` — the client's stale version for reference. (5) `yourData` — what the client tried to write (echoed back). (6) `conflictingFields` — array of field names where both the client and server modified the value. (7) `resolutionHint` — `FIELD_LEVEL_MERGE_POSSIBLE`, `MANUAL_REVIEW_REQUIRED`, or `AUTO_MERGED`. This schema allows the client to implement a diff view

or automatic merge without another GET request.

Q2. How does ETag / If-Match implement optimistic concurrency in REST APIs?

ETag is a version identifier (hash of content, or version number like 'v5') returned in GET responses. The client stores the ETag. On a subsequent PUT/PATCH, the client sends If-Match: 'v5'. The server compares this against the current DB version. If they match, the resource has not been modified concurrently — the write proceeds and the response returns the new ETag. If they don't match, another request modified the resource since the client's last read — the server returns 409 Conflict (or 412 Precondition Failed). This implements optimistic concurrency: reads are free, writes detect conflicts at commit time. The ETag approach is stateless — no server-side session needed, works across load-balanced servers as long as ETags are deterministic.

Q3. How do you map JPA OptimisticLockException to HTTP 409?

JPA @Version annotation on a Long/Integer field causes Hibernate to automatically include the version in UPDATE WHERE clauses: UPDATE booking SET price=X, version=6 WHERE id=Y AND version=5. If no row is updated (version mismatch), Hibernate throws OptimisticLockException (or Spring's ObjectOptimisticLockingFailureException). Handle in @RestControllerAdvice with @ExceptionHandler(ObjectOptimisticLockingFailureException.class) annotated with @ResponseStatus(HttpStatus.CONFLICT). Inside the handler, reload the current entity state from the database to build a rich 409 response body. This approach requires no manual version checking — Hibernate handles it transparently. The handler can extract entity class and ID from ex.getPersistentClassName() and ex.getIdentifer() to fetch the current state.

Q4. Explain last-write-wins vs first-write-wins conflict resolution.

Last-Write-Wins (LWW): whichever write has the highest timestamp or version number wins. The earlier write's data is silently discarded. Simple to implement but dangerous for collaborative data — if Alice edits price and Bob simultaneously edits status, one of their changes will be lost. Acceptable for: sensor data streams, session state, append-only counters. First-Write-Wins: the first write succeeds; all subsequent concurrent writes are rejected with 409 until the client fetches the new version. No data loss — the rejected client must re-fetch and decide. More complex UX (must handle rejection) but safer. This is the optimistic concurrency model implemented by ETag/If-Match. Choice: use LWW only when data loss is acceptable; use first-write-wins for anything requiring data integrity.

Q5. Describe the booking hold pattern and why it prevents double-booking.

The booking hold pattern introduces a HELD intermediate state between AVAILABLE and BOOKED. When a user selects a seat: (1) Server executes SELECT ... FOR UPDATE on the seat row (prevents concurrent hold attempts). (2) If status=AVAILABLE: set status=HELD, held_by=userId, held_until=now()+10min. (3) Client has 10 minutes to complete payment. (4) On payment completion: confirm() checks status=HELD, held_by=userId, held_until > now(); transitions to BOOKED. (5) Background scheduler: UPDATE seats SET status=AVAILABLE WHERE status=HELD AND held_until < now(). Double-booking prevention: the SELECT FOR UPDATE in step 1 serializes concurrent hold attempts — only one succeeds, the other sees status=HELD and throws SeatNotAvailableException. The TTL prevents indefinitely locked seats when users abandon payment.

Q6. What is an idempotency key and how does the deduplication mechanism work?

An idempotency key is a client-generated UUID sent in X-Idempotency-Key header with POST/PUT requests. The server stores the result of processing each unique key in an idempotency_keys table (key, status_code, response_body, expires_at). On receiving a request: (1) Look up the key in the table. (2) If found: return the cached response immediately without re-executing business logic — this handles network retries safely. (3) If not found: process the request, save the result, return it. This ensures that retrying a failed payment (e.g.,

network timeout) does not charge twice. Key design decisions: (a) Key lookup must be atomic with the first processing (use DB unique constraint or Redis SETNX). (b) Expire keys after 24-72 hours. (c) Validate that the request body matches the first request for the same key (reject body mismatches).

Q7. How does 3-way merge work in the context of API conflict resolution?

3-way merge uses three versions: (1) base — the last common ancestor (the version both the client and server diverged from). (2) client — the client's proposed new state. (3) server — the current server state. For each field: if only the client changed it (vs base), accept the client's change. If only the server changed it, accept the server's change. If both changed it to the same value, accept it. If both changed it to different values — true conflict, requires manual resolution. This mirrors git's merge algorithm. Implementation: store the base version ID with each write; on conflict, load base version from a version history table and run the 3-way comparison. Advantage over LWW: non-overlapping concurrent edits are auto-merged without data loss. Disadvantage: requires storing historical versions.

Q8. What is field-level merge and when should you use it over 3-way merge?

Field-level merge resolves conflicts attribute-by-attribute using the base, client, and server states. It is functionally identical to 3-way merge but implemented at the application level using reflection or explicit field comparison logic. Fields changed only by the client use the client value; fields changed only by the server use the server value; fields changed by both with different values are flagged as true conflicts. Use field-level merge when: the resource has many independent fields that users edit in isolation (CRM records, product catalog, user profiles). Use 3-way merge when field values have complex interdependencies (changing one field may invalidate another). Field-level merge is more common in practice because it is simpler to implement than a general diff algorithm and works well for flat JSON resources.

Q9. How do you design a conflict resolution UI for web or mobile clients?

On receiving a 409, the client should: (1) Parse conflictingFields from the response body. (2) Display a split-pane view showing 'Your version' vs 'Current server version' for each conflicting field. (3) For non-conflicting fields (only the client changed), auto-apply without showing them. (4) For each conflicting field: show a radio button or inline editor letting the user pick 'Keep mine', 'Use server's', or 'Edit manually'. (5) Assemble the resolved object from all field choices. (6) Re-submit the PUT/PATCH with the resolved data and the currentVersion ETag from the 409 response. Key UX principles: minimize the number of conflicts shown (auto-merge what is safe); never lose the user's unsaved input; explain in plain language what happened.

Q10. What happens if two concurrent requests both try to hold the same seat?

Both requests enter holdSeat() and issue SELECT ... FOR UPDATE. PostgreSQL grants the lock to one request (say Request A) and blocks Request B. Request A sees status=AVAILABLE, sets it to HELD, and commits, releasing the lock. Request B unblocks, re-reads the row, sees status=HELD, and throws SeatNotAvailableException — which the controller maps to HTTP 409 or 423 Locked. This is correct behavior: only one hold is granted. Without SELECT FOR UPDATE, both could read status=AVAILABLE concurrently, both decide to hold, and both commit — resulting in a double hold. The HELD status + FOR UPDATE combination makes the hold operation atomic and conflict-free. Connection timeout on the FOR UPDATE lock can be configured: SET lock_timeout = '5s' per session.

Q11. How do you handle conflicts in a microservices architecture where the database is remote?

In microservices: (1) The resource-owning service is the single authority for conflict detection — all writes go through it, never directly to the database from other services. (2) API gateway or BFF forwards ETag/If-Match headers to the owning service unchanged. (3) The owning service performs the version check and returns 409 with the conflict body. (4) The calling service propagates the 409 upstream to the client. (5) Distributed scenarios: if a saga involves multiple services updating different resources, each service independently detects conflicts on its own resource. Compensating transactions handle partial saga completion. (6)

Event-driven: for async updates via events, use versioned events (sequence numbers) and idempotency keys to detect duplicate/out-of-order events.

Q12. Explain why 412 Precondition Failed and 409 Conflict are different status codes.

412 Precondition Failed: the server evaluates a conditional header (If-Match, If-None-Match, If-Modified-Since) and the precondition is false. It means 'the precondition you stated was not met'. Appropriate when a client sends If-Match: 'v5' and the server has 'v7' — the precondition evaluation failed. 409 Conflict: the request is logically valid but conflicts with the current state of the resource in a way that requires action by the client. More semantic than 412 — it communicates a business conflict. In practice, many APIs use 409 for optimistic concurrency failures because it allows a richer body explaining the conflict. RFC 7232 technically suggests 412 for If-Match failures, but 409 is widely accepted for application-level conflicts. Consistent API documentation should clarify which code is used for which scenario.

Q13. What is the difference between idempotency and conflict resolution?

Idempotency: ensures that retrying the exact same request (same idempotency key) produces the same result as the first successful execution — used for network retry safety. Conflict resolution: handles the case where two different requests (from different users or sessions) attempt incompatible modifications to the same resource. These are orthogonal concerns: (1) Idempotency key prevents double-processing of the same logical operation on retry. (2) ETag/If-Match detects concurrent modification by different parties. A complete API needs both: an idempotency key ensures a payment isn't charged twice on retry; an ETag ensures that two users editing the same product don't silently overwrite each other. Implement both independently — idempotency in an infrastructure filter, conflict detection in business/service layer.

Q14. How do you test conflict resolution logic in a Spring Boot application?

Integration tests using `@SpringBootTest`: (1) Create a resource, note the ETag. (2) Send a PUT with the ETag — assert 200 and new ETag. (3) Send a second PUT with the OLD ETag — assert 409 with conflict body containing `currentVersion`, `conflictingFields`. (4) Test idempotency: send same POST twice with same `X-Idempotency-Key` — assert both return 200 with identical body. (5) Test hold pattern: hold a seat, hold the same seat again from a different session — assert second hold returns 409/423. (6) Concurrent test: use `CountDownLatch` to fire two threads simultaneously to the same PUT — assert exactly one 200 and one 409. (7) TTL test: mock the clock (use `TestClock`), hold a seat, advance clock past TTL, assert hold is released by scheduler. Use `TestContainers` with a real PostgreSQL for full fidelity.

Q15. What is the `resolutionHint` field in a 409 response and how should clients use it?

`resolutionHint` is a machine-readable enum in the 409 body that tells the client how complex the conflict is: `FIELD_LEVEL_MERGE_POSSIBLE` — the conflicting fields are independent; the client can auto-merge by taking its changes on non-conflicting fields and presenting only the truly conflicting fields for user review. `MANUAL_REVIEW_REQUIRED` — the conflict involves interdependent fields or business rule violations that require human judgment (e.g., changing both price and discount simultaneously in a way that violates a business rule). `AUTO_MERGED` — the server already merged the changes automatically and the body contains the merged result; the client should display it and confirm before resubmitting. Clients parse this field to decide whether to show a full conflict UI, auto-merge silently, or display a pre-merged result for confirmation.

Q16. How do you prevent a race condition in the booking hold pattern when the TTL scheduler runs?

The scheduler must use a `WHERE` clause with a conditional `UPDATE` to atomically release expired holds: `UPDATE seats SET status='AVAILABLE', held_by=NULL, held_until=NULL WHERE status='HELD' AND held_until < now()`. This is atomic — it only releases seats that are still in HELD state and past their TTL. Race condition scenario: (1) Scheduler reads that seat X's TTL expired. (2) User Y's `confirm()` starts and acquires

SELECT FOR UPDATE on seat X. (3) Scheduler's UPDATE runs but the FOR UPDATE lock is held — it blocks. (4) confirm() validates TTL — it's expired, throws HoldExpiredException. (5) confirm() commits, releasing the lock. (6) Scheduler's UPDATE runs — seat is now BOOKED, the WHERE status='HELD' clause excludes it — no rows updated. The combination of SELECT FOR UPDATE in confirm() and the conditional WHERE in the scheduler UPDATE prevents any race.

Q17. How would you implement merge conflict resolution in a mobile offline-first app?

Mobile offline-first apps sync changes to the server when connectivity is restored. Strategy: (1) Track a clientModifiedAt timestamp for each local change. (2) On sync, send all local changes with the last-synced server version (as If-Match). (3) Server runs field-level merge using the last-synced version as the base. (4) For auto-mergeable changes: apply silently, return merged result. (5) For conflicts: return 409 with conflictingFields. (6) Mobile app prompts user to resolve conflicts offline. (7) User's resolution is synced in the next attempt. Additional considerations: use vector clocks or logical timestamps to order operations from multiple devices. Store a local change log (like an event log) so conflicts can be reconstructed. Implement exponential backoff on sync retries. Consider CRDTs for eventually-consistent data types (counters, sets) that auto-merge without conflicts.

Q18. What are the risks of using timestamps for last-write-wins in a distributed system?

Clock skew: different servers have slightly different system clocks (typically 100ms-1s difference without NTP, potentially more with drift). Two events that appear simultaneous or in different order depending on which server's clock is read. LWW based on wall-clock timestamps can incorrectly discard the logically later write if the writing server's clock is behind. Mitigation options: (1) Use a centralized monotonic sequence generator (database sequence, ZooKeeper, Redis INCR) instead of wall clock. (2) Use hybrid logical clocks (HLC) which combine physical time with a logical component to handle skew. (3) Use vector clocks for causal ordering. (4) Use a version number managed by the database (@Version in JPA) — deterministic, no clock skew. Pure LWW with wall clock timestamps is only safe for data where occasional overwrites of genuinely concurrent edits are acceptable (session data, metrics).

Q19. How do you design a conflict-free API for high-concurrency inventory decrement?

Inventory decrement (e.g., selling items) has high conflict potential. Strategies: (1) Conditional UPDATE: UPDATE inventory SET quantity = quantity - :qty WHERE id = :id AND quantity >= :qty. Returns 0 rows if insufficient stock — no optimistic lock needed, the DB handles concurrency atomically. (2) Reserve pattern (like booking hold): reserve N items (subtract from available, add to reserved), process, then confirm or release. (3) Pre-allocated buckets: split inventory across shards (bucket 1: items 1-100, bucket 2: items 101-200) — requests go to different buckets, reducing contention. (4) Decrement events: send decrement events to a Kafka topic; a single consumer applies them serially — no concurrency, no conflicts, but adds latency. For high-throughput e-commerce (flash sales): combine buckets + async confirmation for best results.

Q20. What monitoring and alerting should you implement for conflict resolution?

Metrics to track: (1) conflict_rate: counter of 409 responses per endpoint per minute. A spike indicates contention or a client bug (not sending ETag). (2) hold_expiry_rate: counter of TTL-expired holds — high rate suggests users are abandoning checkout or TTL is too short. (3) idempotency_duplicate_rate: count of requests served from idempotency cache — high rate indicates client retry storms. (4) merge_auto_success_rate vs merge_manual_required_rate: ratio of auto-merged to manual conflicts. Alerting: alert if conflict_rate > X% for a critical endpoint (may indicate a bug). Alert if hold_expiry_rate > Y% (checkout abandonment). Log every 409 with: resource type, conflicting fields, client version, server version, user ID — enables debugging. Dashboards: use Micrometer + Prometheus + Grafana; add @Counted and @Timed on conflict-handling methods.